Para qué es esta guía. En el Módulo 2 aprendiste cada herramienta por separado, siempre sobre los datos del gimnasio. Hoy las juntas, en orden, sobre **tus propios datos**. La guía te lleva paso a paso: qué pregunta responde cada etapa, un diagrama de lo que pasa con la tabla, las funciones que necesitas y una lista de chequeo para saber cuándo puedes pasar a la siguiente.

Al terminar la sesión deberías tener:

1. Un notebook de Colab que va de tu archivo crudo a una tabla limpia y enriquecida, sin editar nunca el archivo original.
2. Una tabla resumen (`groupby`) que responde al menos una de tus preguntas de negocio.
3. Dos o tres gráficos, cada uno con su hallazgo escrito.
4. Una **bitácora de decisiones** (plantilla en la última página): qué problemas encontraste y qué decidiste hacer con cada uno.

Cómo leer los ejemplos de código. Usamos nombres genéricos que debes reemplazar por los tuyos:

`df`	tu tabla tal como la cargaste (cruda).
`limpio`	una copia de `df` sobre la que haces todos los cambios.
`"ciudad"`, `"fecha"`, `"monto"`…	nombres de columna de ejemplo: cámbialos por los nombres reales de tu dataset.
`"id_cliente"`	la columna que identifica cada fila (cliente, venta, producto…), si tu dataset tiene una.

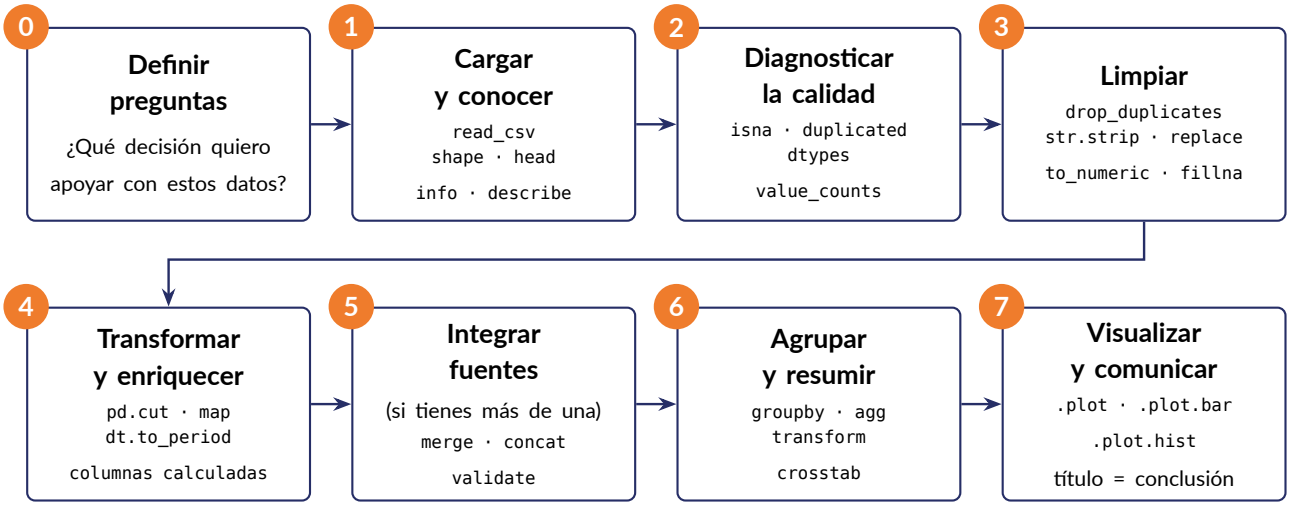
Agenda sugerida para las 3 horas

Hora	Pasos	Producto al final de la hora
Hora 1	0. Preguntas · 1. Cargar · 2. Diagnosticar · 3. Limpiar	Tabla `limpio` sin duplicados, con tipos correctos y nulos tratados.
Hora 2	4. Transformar · 5. Integrar · 6. Agrupar	Columnas nuevas y una tabla resumen por grupo.
Hora 3	7. Visualizar y comunicar	Dos o tres gráficos con hallazgo, y la bitácora completa.

La referencia completa de funciones del curso está en `recursos/referencia-funciones-metodos.pdf`: tenla abierta mientras trabajas.

El mapa completo

Estos son los ocho pasos de la guía. La columna **Clase** de cada tabla de funciones te dice dónde lo viste por primera vez, por si necesitas volver al notebook de esa clase.



Validación continua. Después de *cada* paso cuenta filas (`len`), revisa nulos (`isna().sum()`) y mira los valores (`value_counts()`). Si un número no cuadra, no sigas: vuelve al paso anterior y entiende por qué.

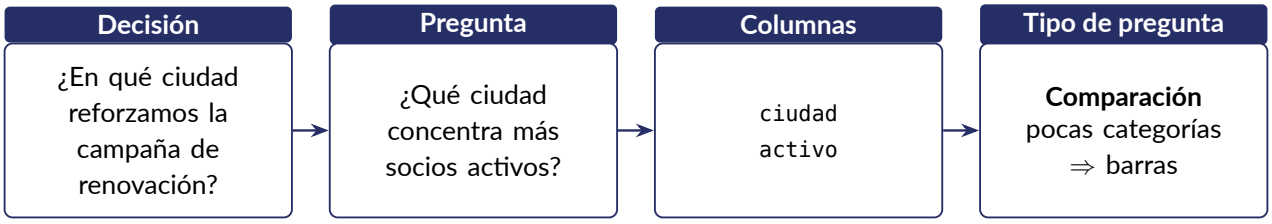
Tres reglas que atraviesan todos los pasos (las viste en la Clase 8):

1. **Nunca edites el archivo crudo.** Todo parte de `pd.read_csv(...)`; si te equivocas, vuelves a leerlo y no perdiste nada.
2. **Trabaja sobre una copia** (`limpio = df.copy()`) y, cuando corrijas una columna, compárala con la original antes de reemplazarla.
3. **Nada que cambie cada vez que lo ejecutas.** Si calculas antigüedades, fija una fecha de referencia (`pd.Timestamp("2026-09-30")`), no uses la fecha de hoy.

0 Definir tus preguntas

Pregunta guía: *¿Qué decisión quiero apoyar, y qué tengo que saber para tomarla?*

Antes de abrir el notebook, escribe dos o tres preguntas de negocio. Cada una te dice qué columnas necesitas y, desde ya, qué tipo de gráfico vas a usar al final (Clase 11). Una pregunta concreta también te dice qué limpiar primero: no hace falta dejar perfecta una columna que no vas a usar.



Ejemplo con los datos del gimnasio. Los cuatro tipos de pregunta son los de la Clase 11: comparación, composición, distribución y relación.

Escribe aquí tus preguntas:

	Pregunta de negocio	Columnas que necesito	Tipo de pregunta
1			
2			
3			

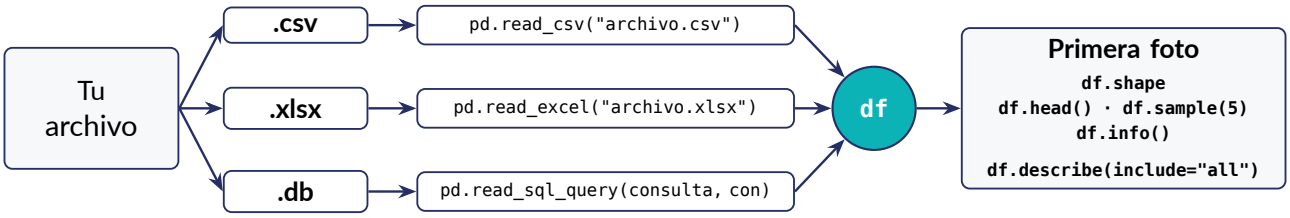
☐ Cada pregunta se puede responder con columnas que existen en mis datos.

☐ Cada pregunta tiene asignado uno de los cuatro tipos.

1 Cargar y conocer tus datos

Pregunta guía: *¿Qué tengo en las manos: cuántas filas, qué columnas y de qué tipo?*

La función de lectura depende del formato del archivo. Todas devuelven lo mismo: un `DataFrame`. A partir de ahí, cuatro órdenes te dan la primera foto de la tabla.



Para…	Función o método	Clase
Leer un CSV o un Excel	`pd.read_csv("archivo.csv")`, `pd.read_excel("archivo.xlsx")`	4, 7
Leer una tabla de una base SQLite	`pd.read_sql_query("SELECT * FROM tabla", con)`	5
Saber el tamaño (filas, columnas)	`df.shape`	7
Ver filas de muestra	`df.head()`, `df.sample(5)`	7
Tipos y valores no nulos por columna	`df.info()`	7
Resumen estadístico (también del texto)	`df.describe(include="all")`	7

En tu notebook

```
import pandas as pd

df = pd.read_csv("mis_datos.csv")
df.shape
df.head()
df.info()
```

Si el archivo no se lee bien

Archivos exportados desde Excel en español suelen usar punto y coma como separador y coma decimal; si todas las columnas aparecen pegadas en una sola, prueba `pd.read_csv("archivo.csv", sep=";", decimal=",")`. Si las tildes salen como símbolos raros, agrega `encoding="latin-1"`.

- ☐ Sé cuántas filas y columnas tiene mi tabla, y qué representa cada fila.
- ☐ Revisé `df.info()`: anoté las columnas cuyo tipo no es el que esperaba.

2 Diagnosticar la calidad

Pregunta guía: *¿Qué problemas tiene la tabla antes de confiar en ella?*

Busca, columna por columna, los cuatro problemas de la Clase 7. El diagrama muestra cómo se ven en una tabla y qué función los delata.

id	ciudad	edad	telefono
1	Medellín	34	3104451122
2	MEDELLIN	treinta	3.031046e+09
3	Bogotá	NaN	3012267788
4	" Medellín"	41	NaN
4	" Medellín"	41	NaN

- **Nulo**: una celda vacía (NaN). `df.isna().sum()`
- **Duplicado**: una fila idéntica a otra. `df.duplicated().sum()`
- **Tipo incorrecto**: texto donde van números. `df.dtypes · pd.to_numeric(..., errors="coerce")`
- **Inconsistencia**: un valor, varias escrituras. `df["ciudad"].value_counts()`

Tabla ilustrativa, con los mismos tipos de problema que encontramos en `membresias_gimnasio.csv` en la Clase 7.

Para...	Función o método	Clase
Contar nulos por columna	`df.isna().sum()`	7
Ver el porcentaje de nulos por columna	`(df.isna().mean() * 100).round(1)`	7
Contar filas exactamente repetidas	`df.duplicated().sum()`	7
Ver si la llave se repite	`df["id_cliente"].duplicated().sum()` o `.is_unique`	7, 10
Ver el tipo de cada columna	`df.dtypes`	7
Encontrar texto disfrazado de número	`pd.to_numeric(df["edad"], errors="coerce")`	7
Ver las variantes de una columna de texto	`df["ciudad"].value_counts()`, `.unique()`	7

En tu notebook: el diagnóstico completo

```
df.isna().sum()                      # nulos por columna
df.duplicated().sum()                # filas repetidas
df.dtypes                            # ¿cada columna tiene el tipo que esperas?

# texto disfrazado de número: se vuelve NaN al convertir, pero no estaba vacío
numero = pd.to_numeric(df["edad"], errors="coerce")
df.loc[numero.isna() & df["edad"].notna(), "edad"]

for columna in ["ciudad", "categoria"]:   # repite con tus columnas de texto
    print(df[columna].value_counts(), "\n")
```

Lo que pasó en el curso

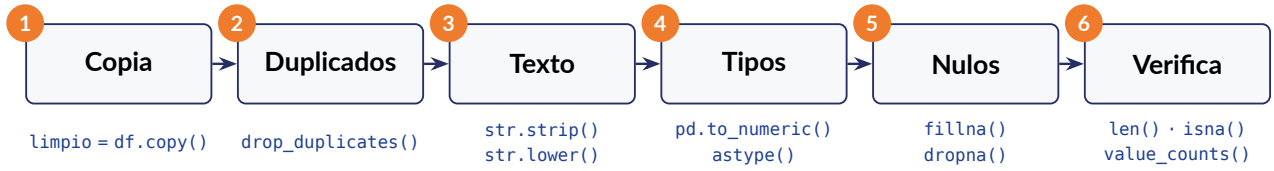
El archivo crudo del gimnasio tenía 54 filas, de las cuales 4 eran duplicados exactos; `ciudad` tenía 12 formas de escribir 4 ciudades (`Medellín`, `Medellin`, `MEDELLIN`, `" Medellín"`, `medellín`...); y `edad` tenía el texto `"treinta"`, que `isna()` nunca habría mostrado porque técnicamente no es un nulo.

- ☐ Para cada columna que uso en mis preguntas, sé si tiene nulos, tipo incorrecto o inconsistencias.
- ☐ Sé si hay filas duplicadas y si mi llave (si la hay) es única.
- ☐ Anoté cada problema en la bitácora, antes de corregir nada.

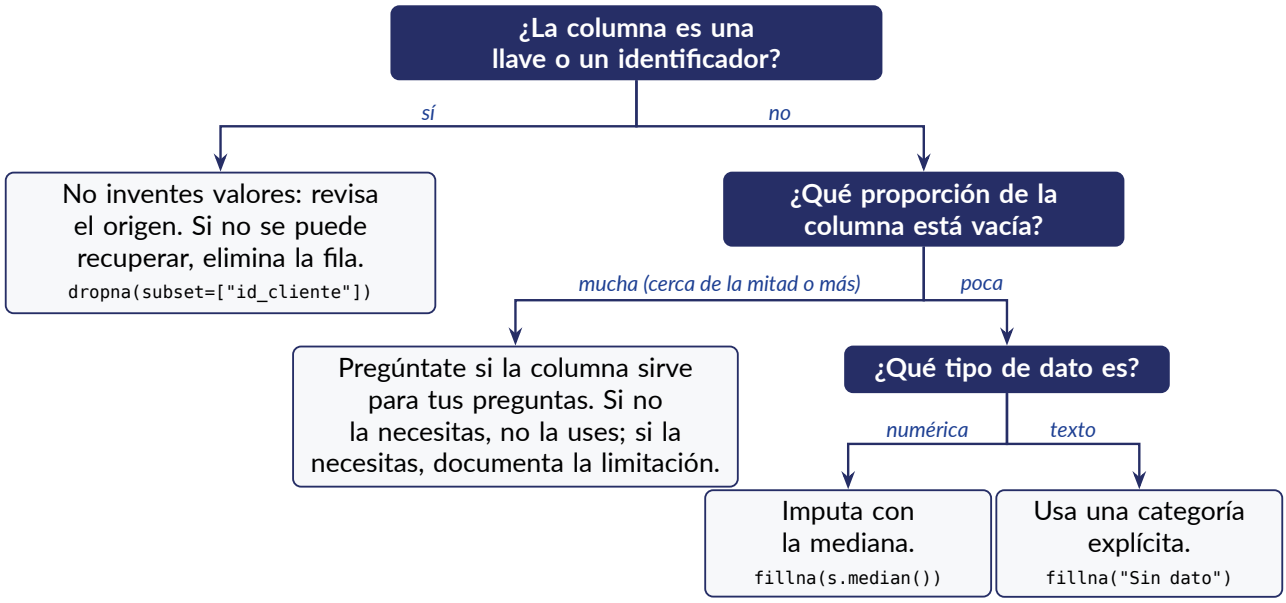
3 Limpiar

Pregunta guía: *¿Qué corrijo, en qué orden, y qué hago con lo que no se puede corregir?*

El orden importa. Los duplicados van primero, porque si no, cada conteo y cada promedio posterior queda inflado por las copias. Los nulos van al final, porque una conversión de tipo puede crear nulos nuevos (`errors="coerce"`).



¿Qué hago con los nulos? No hay una única respuesta correcta: hay una decisión que tomar, y que debe quedar escrita. Este árbol resume las decisiones que discutimos en la Clase 8.



Para...	Función o método	Clase
Quitar filas repetidas	`limpio.drop_duplicates()`	8
Quitar espacios y unificar mayúsculas	`.str.strip()`, `.str.lower()`, `.str.title()`	8
Unificar variantes que quedan (tildes)	`.replace({"medellin": "medellín"})`	8, 10
Convertir texto a número	`pd.to_numeric(s, errors="coerce")`	7, 8
Convertir a texto (códigos, teléfonos)	`s.astype(str)`	8
Rellenar nulos	`s.fillna(s.median())`, `s.fillna("Sin dato")`	8
Eliminar filas con nulos en columnas clave	`limpio.dropna(subset=["id_cliente"])`	12

En tu notebook

```
limpio = df.copy()
limpio = limpio.drop_duplicates()

limpio["ciudad_limpia"] = (limpio["ciudad"].str.strip()
                                           .str.lower()
                                           .replace({"medellin": "medellín", "bogota": "bogotá"})
                                           .str.title())
limpio["ciudad_limpia"].value_counts()      # compara con la original antes de reemplazar

limpio["edad"] = pd.to_numeric(limpio["edad"], errors="coerce")
print("edades a imputar:", limpio["edad"].isna().sum())   # anótalo en la bitácora
limpio["edad"] = limpio["edad"].fillna(limpio["edad"].median())

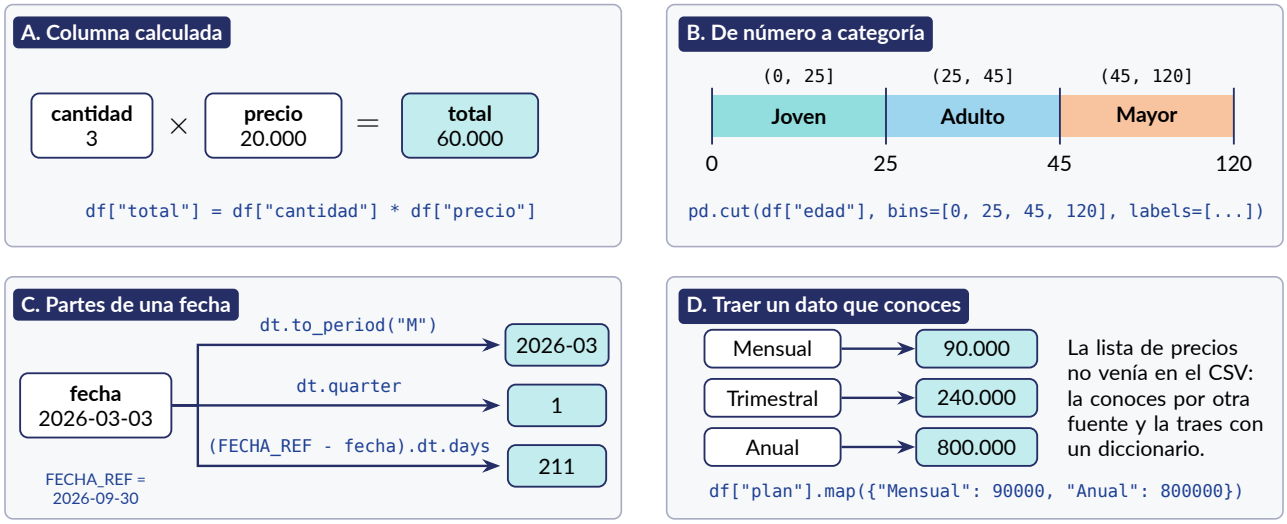
print(len(df), "->", len(limpio))
```

- ☐ No quedan duplicados, y sé cuántas filas eliminé.
- ☐ Cada columna de texto que uso tiene solo valores válidos (revisado con `value_counts()`).
- ☐ Las columnas numéricas son números y las fechas son fechas (`limpio.dtypes`).
- ☐ Para cada columna con nulos tomé una decisión, y escribí cuántos valores imputé o eliminé.

4 Transformar y enriquecer

Pregunta guía: *¿Qué columnas nuevas necesito para responder mis preguntas?*

Las columnas originales casi nunca son las que necesitas. Cuatro transformaciones cubren la mayoría de los casos:



Ejemplos de las Clases 8 y 9: la fecha y la antigüedad son las de un socio real del dataset; los precios, los de la lista del gimnasio.

Para…	Función o método	Clase
Calcular una columna a partir de otras	`df["c"] = df["a"] * df["b"]`	4, 8
Convertir un número en categorías	`pd.cut(s, bins=[...], labels=[...])`	8
Convertir texto a fecha	`pd.to_datetime(df["fecha"])`	8, 9
Mes, trimestre, días transcurridos	`.dt.to_period("M")`, `.dt.quarter`, `.dt.days`	8, 9
Reemplazar cada valor según un diccionario	`s.map({...})`	8
Regla que usa varias columnas a la vez	`limpio.apply(funcion, axis=1)`	8

En tu notebook

```
FECHA_REF = pd.Timestamp("2026-09-30")          # fija: no uses la fecha de hoy

limpio["fecha"] = pd.to_datetime(limpio["fecha"])
limpio["mes"] = limpio["fecha"].dt.to_period("M")
limpio["antiguedad_dias"] = (FECHA_REF - limpio["fecha"]).dt.days

limpio["grupo_edad"] = pd.cut(limpio["edad"], bins=[0, 25, 45, 120],
                              labels=["Joven", "Adulto", "Mayor"])
limpio["grupo_edad"].value_counts()             # ¿quedó alguien sin grupo (NaN)?
```

Primero lo vectorizado

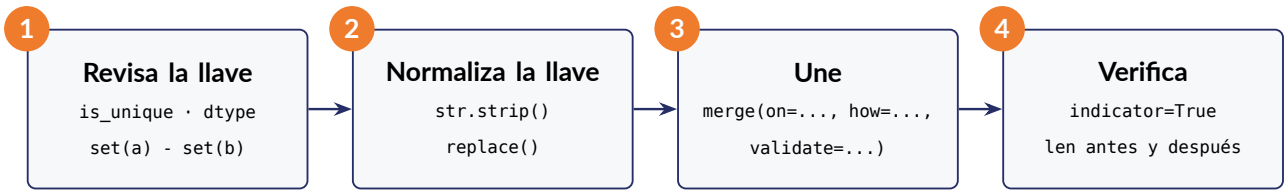
Si existe una operación que trabaja sobre la columna completa (`+`, `*`, `.str`, `.dt`, `pd.cut`, `.map`), úsala. Reserva `.apply(…, axis=1)` para reglas que necesitan varias columnas de la misma fila al mismo tiempo.

- ☐ Cada columna nueva responde a algo que necesito para mis preguntas.
- ☐ Revisé cada columna nueva con `value_counts()` o `describe()`, y no generó nulos inesperados.

5 Integrar fuentes (si tienes más de una)

Pregunta guía: *¿Cómo junto mis tablas sin perder ni duplicar filas en silencio?*

Si todos tus datos están en una sola tabla, puedes saltar este paso. Si tienes un catálogo (productos, sedes, planes) o archivos que llegan por partes, la integración tiene cuatro momentos, y el tercero es el único que escribe la unión.



El parámetro `how` decide qué filas sobreviven (verde: se conserva; gris: se descarta). En la mayoría de los análisis quieres `how="left"`: partir de tu tabla principal y agregarle información sin perder a nadie.



Para...	Función o método	Clase
Comprobar que la llave no se repite en el catálogo	`tabla["ciudad"].is_unique`	10
Ver valores sin pareja antes de unir	`set(limpio["ciudad"]) - set(tabla["ciudad"])`	10
Unir por una llave, validando la relación	`limpio.merge(tabla, on="ciudad", how="left", validate="many_to_one")`	10
Detectar lo que quedó sin pareja	`merge(..., indicator=True)` y `["_merge"].value_counts()`	10
Apilar tablas con las mismas columnas	`pd.concat([a, b], ignore_index=True)`	10

En tu notebook

```
print(tabla["ciudad"].is_unique)
print(set(limpio["ciudad_limpia"]) - set(tabla["ciudad"]))    # debería quedar vacío

unida = limpio.merge(tabla, left_on="ciudad_limpia", right_on="ciudad",
                     how="left", validate="many_to_one", indicator=True)
print(unida["_merge"].value_counts())        # ¿cuántos left_only?
print(len(limpio), "->", len(unida))        # en un left many_to_one deben ser iguales
```

Lo que pasó en el curso

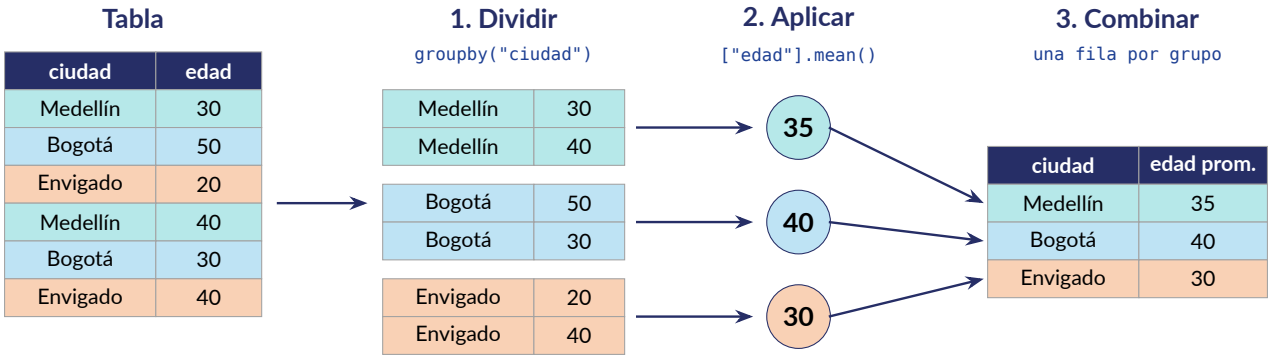
En la Clase 10, la sede de Itagüí estaba escrita `"Itagui"` y sus socios quedaban sin sede sin ningún error; al apilar las nuevas inscripciones, el `id_socio` 7 apareció dos veces con dos personas distintas; y una sede repetida en el catálogo convirtió 50 socios en 72 filas. Las tres cosas se detectan con los chequeos de esta página.

- ☐ La tabla unida tiene el número de filas que esperaba, y sé por qué.
- ☐ Revisé los `left_only` y `right_only`, y anoté qué hice con ellos.
- ☐ Usé `validate` en cada `merge`.

6 Agrupar y resumir

Pregunta guía: *¿Cómo se comporta cada grupo, comparado con los demás?*

`groupby` hace siempre lo mismo, en tres tiempos: divide la tabla en grupos, aplica un cálculo a cada uno y junta los resultados en una tabla nueva, con una fila por grupo.



Datos ilustrativos (seis filas) para mostrar la mecánica.

Para…	Función o método	Clase
Contar filas por grupo	`limpio.groupby("ciudad").size()`	9
Varios indicadores a la vez, con nombre	`.agg(clientes=("id_cliente", "count"), monto=("monto", "sum"))`	9
Ordenar el resumen	`.sort_values("clientes", ascending=False)`	9
Tabla cruzada de dos categorías	`pd.crosstab(limpio["ciudad"], limpio["plan"])`	9
Total del grupo en cada fila (para participaciones)	`.groupby("ciudad")["monto"].transform("sum")`	9
Resumen por mes	`limpio.groupby("mes").size()`	9

En tu notebook

```
resumen = (limpio.groupby("ciudad_limpia")
                 .agg(clientes=("id_cliente", "count"),
                      monto_total=("monto", "sum"),
                      edad_promedio=("edad", "mean"))
                 .sort_values("clientes", ascending=False)
                 .round(1))
resumen["participacion"] = (resumen["clientes"] / resumen["clientes"].sum() * 100).round(1)
resumen
```

Un promedio sin su conteo engaña

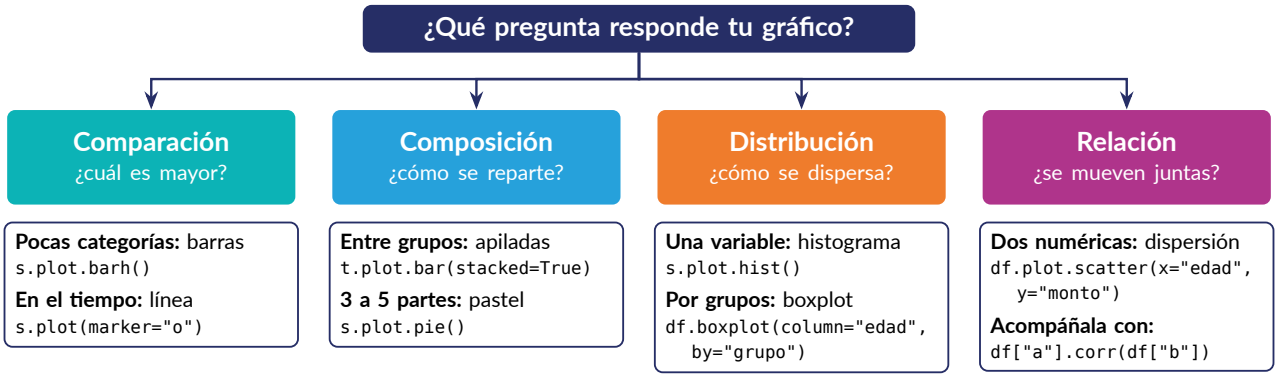
Incluye siempre una columna con el número de filas de cada grupo. En la Clase 9, Itagüí tenía solo 2 socios: su promedio se veía igual de "serio" que el de Medellín, con 22. Si un grupo tiene muy pocas filas, dilo junto al resultado o júntalo con otros.

- ☐ El resumen responde directamente a una de mis preguntas del Paso 0.
- ☐ Tiene una columna de conteo, está ordenado y redondeado.
- ☐ La suma de los conteos coincide con el total de filas de `limpio`.

7 Visualizar y comunicar

Pregunta guía: *¿Qué gráfico responde mi pregunta, y qué conclusión quiero que se lleve quien lo vea?*

Vuelve a tus preguntas del Paso 0: el tipo de pregunta decide el gráfico. Todos se hacen con el método `.plot` de pandas, directamente sobre la columna o la tabla resumen.



`s`: una columna o un resumen (`Series`); `t`: una tabla cruzada; `df`: tu tabla. Ejemplos completos en el notebook de la Clase 11.

En tu notebook: un gráfico completo

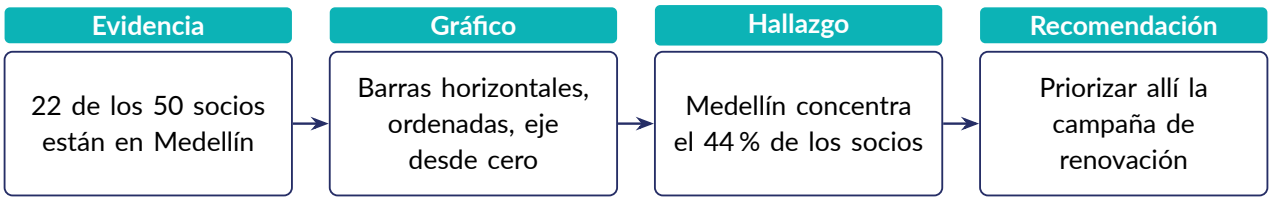
```
import matplotlib.pyplot as plt

resumen["clientes"].sort_values().plot.barh(color="#0CB3B6",
                                            title="Escribe aquí tu conclusión, no una descripción")
plt.xlabel("Clientes")
plt.show()
```

Antes de mostrarlo (la lista de la Clase 11):

- ☐ El título dice la conclusión (“Medellín concentra el 44 % de los socios”), no la descripción (“Socios por ciudad”).
- ☐ En barras, el eje empieza en cero.
- ☐ Los ejes tienen nombre y unidad, y las categorías están ordenadas por valor.
- ☐ Cada color significa algo; no hay rejillas, leyendas ni bordes que sobren.
- ☐ Sin doble eje: si son dos variables con unidades distintas, van en dos gráficos.

Del gráfico al hallazgo. Un gráfico no es todavía un hallazgo. El hallazgo es la frase que alguien que toma decisiones puede usar:



Cifras reales del dataset limpio del gimnasio; la recomendación es un ejemplo.

Entregable de la sesión

Al final de la clase, comparte tu notebook y esta página completa. Es también el punto de partida de la Clase 13: el archivo limpio que guardes hoy es el que vas a cargar en Power BI.

Última celda de tu notebook

```
limpio.to_csv("mis_datos_limpios.csv", index=False)    # descárgalo desde el panel de archivos
```

Mis hallazgos

	Evidencia (cifra y gráfico)	Hallazgo	Recomendación
1			
2			
3			

Bitácora de decisiones

Una fila por cada problema que encontraste. La primera es un ejemplo del curso; quien lea tu análisis debe poder entender, solo con esta tabla, qué cambiaste en los datos y por qué.

Paso	Qué encontré	Qué decidí	Filas afectadas	Por qué
3	*`edad` tenía texto (`"treinta"`) y celdas vacías*	*Convertir con `to_numeric` e imputar con la mediana*	*las que no eran un número*	*La mediana no se deja arrastrar por valores extremos*